



Trabajo Práctico Tolerancia

Sistemas Distribuidos I (TA050)

1° Cuatrimestre 2026

Alumno	Padrón	E-mail
Ignacio Ezequiel Vetrano	106129	ivetrano@fi.uba.ar
Manuel Rivera Villatte	106041	mriverav@fi.uba.ar
Ezequiel Lazarte	108063	ezlazarte@fi.uba.ar

Arquitectura General del Sistema

El sistema es un pipeline de procesamiento de datos en *streaming* diseñado mediante un gráfico de robustez. Su objetivo es procesar transacciones bancarias resolviendo 5 requerimientos (flujos) de análisis en paralelo. Para optimizar recursos y evitar el procesamiento redundante, el sistema bifurca tempranamente las transacciones según su moneda (USD vs. Resto) y fechas, reutilizando nodos de limpieza y filtrado.

A continuación, se detalla el ciclo de vida de los datos desde su ingreso hasta la consolidación de resultados:

1. Ingreso y Bifurcación Principal

- **Gateway:** Recibe el dataset del cliente vía TCP/IP y lo inyecta al sistema a través de una cola de entrada compartida.
- **Cleaner Inicial (Stateless):** Un grupo de workers lee las transacciones en Round-Robin y elimina columnas innecesarias para todo el sistema (como `Amount_Received`, `Receiving_Currency` e `Is_Laundering`).
- **Router Principal (Exchange):** Toma las transacciones limpias y las bifurca en dos grandes ramas:
 - Rama `tx.usd`: Exclusiva para transacciones originadas en Dólares.
 - Rama `tx.*`: Recibe todas las transacciones sin importar su moneda.

2. Flujo 1: Filtro Simple (Transacciones USD < 50)

- Toma datos directamente de la rama `tx.usd`.
- Pasa por un **Cleaner** que descarta campos irrelevantes (`Timestamp`, `Payment_Format`, etc.).
- Un nodo **Amount Filter (Stateless)** evalúa cada transacción y deja pasar únicamente aquellas cuyo monto (`Amount_Paid`) sea menor a 50.
- Los resultados viajan directamente al nodo unificador final (**Join**).

3. Flujo 2: Máximo Transferido por Banco

- También se alimenta de la rama `tx.usd`.
- Tras una etapa de **Cleaner**, las transacciones llegan a un **Router (Stateless)**.
- Este Router envía los datos a un *Exchange* ruteando mediante un **hash del Banco de origen** (`hash[from_bank]`).
- Los datos caen en nodos **Max (Stateful)**. Gracias al ruteo por hash, todas las transacciones de un mismo banco caen en el mismo contenedor. Este nodo mantiene en memoria el monto máximo histórico visto por banco. Al recibir un EOF, flushes sus máximos locales.
- Un **Join (Stateful)** final consolida estos resultados.

4. Flujo 3: Filtro Dinámico por Promedio Histórico

Este flujo tiene una dependencia temporal estricta y se divide en dos fases que corren en paralelo pero se sincronizan al final:

- **Fase A (Cálculo del Promedio):**
 - Toma datos de la rama general (tx.*) y los pasa por un **Date Filter** que aísla el período [09-01 a 09-05]. De este subconjunto, se filtra nuevamente para quedarse solo con las originadas en USD.
 - Se rutea por hash[Payment_Format] hacia nodos **Aggregator (Stateful)** que acumulan cantidades y montos por formato de pago.
 - Al recibir el EOF, calculan el promedio y lo difunden (Broadcast) mediante un *Fanout Exchange* hacia una cola de control (q_computed_avg).
- **Fase B (Filtrado de Transacciones):**
 - Toma datos de la rama directa de tx.usd.
 - Un **Date Filter** aísla el período [09-06 a 09-15].
 - Estas transacciones se encolan en un buffer persistente. Los nodos **Amount Filter (Stateful)** responsables de procesar esta cola operan con una barrera de sincronización: no comienzan a evaluar datos hasta recibir el mensaje con el promedio calculado desde la Fase A.
 - Una vez recibido el promedio, filtran las transacciones evaluando si son mayores al mismo y envían el resultado al **Join**.

5. Flujo 4: Detección de Patrones Scatter-Gather

Busca identificar transferencias de una cuenta A hacia una cuenta C utilizando una cuenta puente B.

- Se alimenta del bloque de transacciones ya filtradas por fecha [09-01 a 09-05].
- Un **Cleaner** se asegura de conservar la tupla compuesta (Bank, Account) para identificar unívocamente las cuentas de origen y destino.
- Un **Router (Stateless)** lee cada transacción (Origen -> Destino) y genera dos mensajes separados:
 1. Mensaje tipo **SCATTER** ruteado por hash(Origen).
 2. Mensaje tipo **GATHER** ruteado por hash(Destino).
- Estos mensajes llegan a nodos **Aggregator (Stateful)** que construyen grafos parciales en memoria. Al recibir el EOF, realizan un cruce (Join local) buscando coincidencias exactas donde una cuenta actuó como origen en un mensaje y destino en otro (la cuenta puente).
- Los caminos encontrados se rutean por hash hacia un **Accumulator (Stateful)** que cuenta las ocurrencias de cada ruta.
- Un **Value Filter (Stateless)** final deja pasar únicamente las rutas que se repitieron 5 o más veces, enviándolas al **Join**.

6. Flujo 5: Conteo de Micro-transferencias Internacionales

Busca contar transacciones tipo Wire/ACH cuyo valor final convertido a dólares sea menor a 1.

- Toma transacciones generales del filtro de fechas [09-01 a 09-05].
- Un **Format Filter (Stateless)** descarta todo lo que no sea Wire o ACH.

- Se rutean los mensajes hacia la etapa de conversión. Aquí existe una optimización de diseño (Bypass): las transacciones cuya moneda ya es USD saltan directamente al filtro final.
- Las transacciones en otras monedas (`tx.non-usd`) se rutean por `hash(Currency)` a nodos **Conversor USD (Stateful)**. Estos nodos mantienen un caché local de cotizaciones; si no tienen el dato, consultan a una API externa, lo guardan y realizan la conversión de moneda.
- Todas las transacciones (las convertidas y el bypass de USD originales) confluyen en un **Amount Filter (Stateless)** que descarta los montos ≥ 1 .
- Un **Join (Stateful)** cuenta la cantidad de transacciones resultantes y consolida el total.

7. Envío Final

El último nodo de cada flujo se encarga de enviar el resultado obtenido al gateway, y este se encarga de, a medida que llegan las respuestas de los diferentes flujos, enviarlos al cliente.

Workers

Filter

La misión del filter es sencilla: recibir un paquete de transacciones, y forwardear un nuevo paquete solamente con aquellas transacciones que pasen el filtro. El criterio del filtro depende de su configuración, y por lo general consiste de un valor de la transacción (la columna por la que filtrar), un operador y un valor umbral.

Para aprovechar que las queries 3, 4 y 5 necesitan filtrar por la misma fecha, pero la query 5 necesita todas las transacciones mientras que las 3 y 4 solamente las de dolares, este worker especial publica con el topic `tx.usd` o `tx.non-usd` para satisfacer este requisito.

Cleaner

Los workers Cleaner se ocupan de limpiar de las transacciones aquellos valores que ya no se usen a lo largo de una determinada query.

Router

Los Routers están para shardear transacciones a otro worker determinado downstream de acuerdo a un criterio.

Reciben por configuración el valor de la transacción por el que deben enrutar, y forman un hash de ese valor y lo distribuyen a alguno de los siguientes workers de forma determinística.

Un Router particular, el Splitter de la query 4, recibe un batch pero envía dos. Una shardeado por la cuenta de origen, y otra por la cuenta de destino.

Aggregator

El aggregator recibe paquetes de transacciones y las almacena de acuerdo a una función de agregación. Puede ser Max, donde acumula la transacción de mayor valor para cada banco, o Avg, donde acumula el promedio de monto de transacción para cada formato, o simplemente un Count, para la query 5.

La query 4 presenta dos aggregators diferentes:

ScatterAndGather (llamado “aggregator” en la configuración): recibe transacciones en modo Scatter o modo Gather.

Si las recibe en modo Scatter, agrega la cuenta de destino a la lista de cuentas destino de la cuenta origen.

Si las recibe en modo Gather, vice-versa, almacena la cuenta origen en la lista de cuentas origen de la cuenta destino.

Es decir: los workers arman las adyacencias de las cuentas que van recibiendo. El sharding que realiza el Splitter nos asegura que cada worker es “dueño” de las cuentas que reciben, conociendo la totalidad de las entradas y salidas de cada cuenta.

Estos **Aggregators** luego utilizan un exchange (Heavy Accounts Exchange) para que las demás réplicas conozcan las cuentas pesadas: aquellas cuentas que hayan enviado o recibido de 5 cuentas o más.

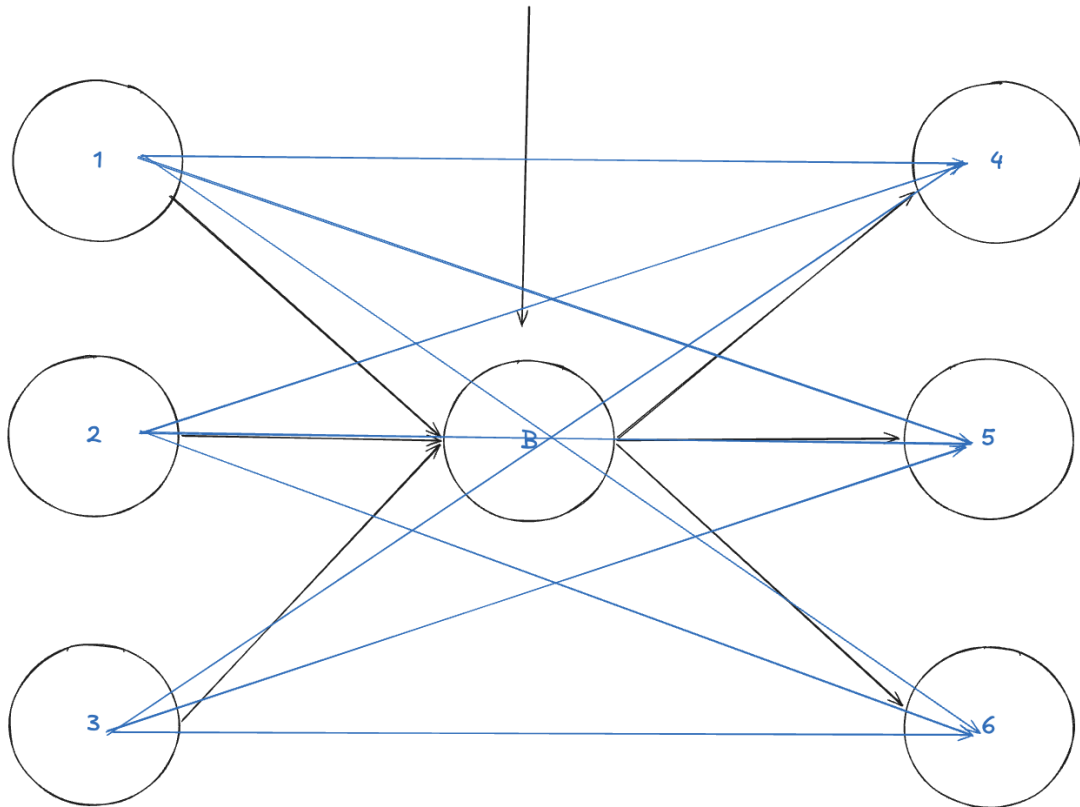
Luego, al recibir EOF, los workers van a juntar los sets de entrada y salida para una cuenta que conocen. Ésta cuenta funciona como “puente”. Las cuentas que enviaron y que recibieron de la cuenta puente son una pareja. El conocimiento general a través de réplicas de las cuentas pesadas permite que los workers ignoren aquellas parejas donde al menos una de las dos no es pesada.

Por ejemplo: si la cuenta 1 envió dinero a la cuenta B, y la cuenta B envió dinero a la cuenta 2, entonces (1, 2) es una pareja con una distancia de 1. Pero queremos ignorar esta pareja si 1 no es una “fuente pesada”, o 2 no es un “sumidero pesado”. Y como otra réplica puede ser dueña de 1 o 2, es necesario que las listas de cuentas pesadas se vuelvan generales.

En el gráfico de aquí debajo, 1, 2 y 3 son cuentas que enviaron dinero a B, y 4, 5 y 6 son cuentas que recibieron dinero de B. El worker va a juntar cada cuenta con cada otra, pero enviar solamente aquellas parejas donde tanto la de origen sea una fuente pesada y el destino sea un sumidero pesado. Y justamente B es una cuenta que este worker conoce por completo. Si otras réplicas forman parejas donde origen o destino fueran B, este sistema permite que puedan actuar acorde a los grados de entrada o salida que conoce la réplica del diagrama.

El siguiente Aggregator, **ScatterGather** (llamado “accumulator” en la configuración) simplemente suma las apariciones de las parejas que recibe.

Conocemos #(input) y
#(output) de esta cuenta



Join

El join de la query 2 se ocupa de juntar transacciones con el nombre del banco de origen. Para esto, consume de dos inputs distintos: la queue de Accounts y del exchange de transacciones que publican los aggregators de la query 2, que buscan las transacciones de mayor monto para cada banco.

El Join va a esperar a recibir todas las accounts antes de consumir transacciones, para asegurar una serialización determinística de los mensajes emitidos.

Converter

Los converters reciben un batch de transacciones y para cada una convierte el monto a dólares. Permite hacer fetch de una conversión mediante la API de Frankfurter.

Para validar correctamente con los datos del notebook de la cátedra, también están hardcodeadas las tasas de conversión, y se pueden utilizar para asegurarse que los resultados sean idénticos a las del notebook. En el código está por defecto que si la divisa

existe en la lista hardcodeda se use esa en lugar de fetchear. Una deuda que queda es configurar esto, para decidir previamente a la ejecución si queremos usar la API o las tasas hardcodedas.

Manejo de EOF

El cliente al terminar de enviar todas las transacciones envía un EOF al sistema para que pueda hacer los cálculos que requieren todas las transacciones.

Al recibir el EOF, el Gateway, que estuvo contando todos los mensajes del cliente, envía al exchange de entrada *e_entry* un EOF con la cantidad de mensajes que recibió de ese cliente, lo que hace que este se propague por todo el sistema de la misma manera que lo hacen las transacciones.

En los Workers el manejo de los EOFs es diferente dependiendo de si en cuyo cluster reciben todos de la misma queue, o está recibiendo cada uno en su propia queue porque la etapa anterior rutea.

Para los casos donde la etapa anterior rutea, el caso es simple, reciben el EOF, hacen flush de los datos que tienen a la siguiente etapa y luego envían el EOF. Los workers de la siguiente etapa esperan a recibir la cantidad de EOFs correspondiente a la cantidad de workers de la etapa anterior.

Para los workers en cuyo cluster reciben todos de la misma queue hay que sincronizar, para eso se realizó un algoritmo de sincronización para este tipo de workers que delegan en el *syncEofController*. Para ello cada worker tiene su propia queue de sincronización por la cual se comunican entre los workers del mismo cluster. El funcionamiento es el siguiente:

- El worker que obtenga el EOF de la queue se convierte en el *Líder*.
- El *Líder* les envía un *AmountRequest* a todos los workers consultando cuantos mensajes recibieron y cuantos enviaron.
- Los workers le responden al Líder con un *AmountResponse* con la información solicitada.
- El Líder espera a recibir la información de todos los workers del cluster, si la cantidad de mensajes recibidos es igual a la cantidad esperada por lo que dice el EOF, entonces se determina que el cluster ya proceso todos los mensajes del cliente, si no coinciden vuelve a intentar, con un sleep que va creciendo en cada reintento.
- Si ya se procesaron todos los mensajes recibidos, el Líder manda EOF a la siguiente etapa, y luego les dice a todos los workers que borren los datos del cliente.

Tolerancia a Fallas

Idempotencia de mensajes, deduping

Como consecuencia de un entorno donde ocurren fallas, es posible que un worker muera entre que emite un mensaje al siguiente worker downstream, y realiza el ack de ese mismo mensaje, resultando en ese mensaje siendo reemitido por el middleware, y por

consecuencia, el mensaje se re-emite al worker downstream. Por lo tanto, el siguiente worker debe tener una forma de deduplicar mensajes que ya recibió y procesó: especialmente aquellos workers cuya lógica de negocio no es idempotente por defecto (por ejemplo, un aggregator que cuenta la cantidad de transacciones que recibe).

Para esto, el gateway stampa un id en cada mensaje que envía al sistema. Estos mensajes deben distinguir de qué cliente se trata, qué stream de datos (cuentas, o transacciones), y el número de secuencia del mensaje.

Los workers generan ids de mensajes de dos formas. Si simplemente re-emiten mensajes a medida que los van procesando, los ids de estos mensajes son un derivado del id recibido. Si en lugar de eso acumulan estado esperando a haber recibido todos los mensajes (como es el caso de los aggregators), se generan nuevos mensajes, indicando la etapa en que se generaron (para que los workers que consumen de más de un input, como es el caso del último filter de la query 3, puedan identificar qué mensajes llegaron del input de transacciones normales y cuales llegaron de la etapa de aggregation que envió los promedios).

Persistencia en disco

Los workers deben persistir en disco toda la información necesaria para operar. Puede ser estado que tenga que ver con la lógica de negocio, o bien la lista de mensajes vistos, o bien rastreo de mensajes recibidos y enviados para validar que la cuenta de transacciones sea la esperada al recibir EOF.

Hay dos formas de persistir un archivo: de manera **Overwrite** (donde los archivos se sobrescriben en su totalidad mediante `os.Rename()`), y **Append** (donde los archivos se abren con la flag `os.O_APPEND`, para escribir directamente al final del archivo).

Los archivos Overwrite se persisten en dos etapas: **Stage** y **Promote**, que involucran dos archivos, uno temporal y uno definitivo. En la etapa Stage se escribe un archivo temporal con los nuevos datos a persistir. Al ejecutar Promote, se realiza un rename del archivo temporal al archivo definitivo.

En cada iteración de este proceso, se genera un número de generación que identifica de forma unívoca al commit que se está realizando. Los archivos Overwrite tienen un encabezado que indica el número de generación con el que fueron escritos. Los archivos Append embeben en cada registro el número de generación correspondiente a su escritura. Esto es porque ambos tipos de archivos tienen distintas estrategias para recuperarse, que explicaremos brevemente.

El orden de persistencia es el siguiente: primero se crea el número de generación (un incremento del anterior). Luego, se realiza Stage de los archivos Overwrite que se tengan que persistir (con el encabezado del nuevo número de generación). Luego, se realiza el Append al archivo de mensajes vistos (con el nuevo número de generación). Luego se realiza el Promote de los archivos temporales, para que queden como el estado final.

Este ordenamiento establece al archivo de mensajes vistos (dedup set) como la fuente de verdad que almacena el último número de generación válido. No se almacena en un archivo aparte, vive dentro del archivo de mensajes vistos.

Veamos cómo se recupera el estado luego de un restart. El archivo Append se carga e identifica el último número de generación válido. Cada registro en el archivo append tiene un checksum (usando la librería hash/crc32) que permite verificar si un registro sufrió una interrupción durante su escritura.

En primer lugar el archivo de mensajes vistos es reconstruido, junto con el último número de generación persistido. Luego, se recuperan los archivos Overwrite, aprovechando el encabezado con el último número de generación: con un simple llamado a Promote, se restaura el archivo temporal siempre y cuando su número de generación sea el último número de generación persistido.

De no ser el caso, no se realiza el Rename, y el anterior archivo definitivo es el que se utiliza para restaurar el estado.

Coordinator

Para abstraer a los workers de este sistema, tenemos un struct llamado Coordinator, que se ocupa de orquestar los checkpoints y las recuperaciones. Exponen a los workers una interfaz sencilla que consiste de:

- **Track(clientID, ack):** Luego de procesar un mensaje. Incrementa un contador interno de mensajes recibidos y almacena la función ack de ese mensaje. Luego de un cierto intervalo de mensajes procesados, llama a Flush().
- **Flush():** Para cada cliente, crea al nuevo número de generación, obtiene el estado a persistir a partir de los Checkpointables (lista de mensajes vistos, estado del worker, lista de mensajes rastreados por EOF) y llama a Stage, Append, Promote. Luego de una persistencia exitosa, llama a todos los acks almacenados uno a uno.
- **Recover():** Recupera el último número de generación, restaura los estados de los Checkpointables a partir de los archivos.
- **Delete(clientID):** Borra los archivos correspondientes a un cliente.
- **SaveClient(clientID):** Persiste el estado de un cliente en un momento particular, solamente utilizado por los aggregators ScatterAndGather de la query 4.

Interfaz Checkpointable

El Coordinator llama a los **Checkpointables**, que son de dos tipos, Overwrite y Append.

Los **Overwrite** implementan los siguientes métodos:

- **SnapshotClient(clientID):** devuelve el estado a persistir en bytes.
- **RestoreClient(clientID, data):** a partir de data (bytes) reconstruye su estado.

Cada estado a persistir/recuperar sabe cómo serializar y deserializar su estado.

El **AppendSource** (Checkpointable modo Append) implementa tres métodos:

- **DrainClient(clientID)**: Devuelve el estado en memoria que todavía no fue persistido. Es decir, el estado acumulado desde el último checkpoint.
- **CommitClient(clientID)**: Borra el estado en memoria correspondiente al fragmento de estado por checkpointear.
- **ReplayClient(clientID, record)**: Regenera el estado, ingestando un registro.

El **Coordinator** llama a **DrainClient** para obtener los datos a appendear, luego llama a **CommitClient** luego de hacer el Append de forma exitosa (antes no, no sería seguro borrar estado en memoria antes de persistir), y al recuperarse llama **ReplayClient** con cada registro recuperado del archivo Append.

Nota: una excepción para el worker ScatterAndGather de la query 4: es el único worker con estado de negocio que crece de forma no acotada (arma listas de adyacencias de cada cuenta que recibe). No es eficiente guardar este estado de forma Overwrite. Por lo tanto, éste estado se almacena usando el método Append, lo que hace que la complejidad temporal de realizar el checkpoint dependa del tamaño del append (los datos a agregar, acumulados desde el anterior checkpoint) en lugar de todo el tamaño del estado.

Limpieza de Clientes

Los archivos de los clientes, tanto como su estado en memoria, es borrado por los workers una vez que se sabe que no se recibirán más mensajes de ese cliente, es decir, una vez que todos los ack sean enviados incluyendo el del EOF de transacciones.

En otras palabras, un procesamiento correcto de un EOF de un cliente limpia los datos de ese cliente en toda la pipeline. Aprovechando esto, si el Gateway detecta que un cliente fue desconectado, simplemente va a propagar un EOF a la pipeline y descartar los resultados recibidos.

Monitores

El sistema tiene N instancias de monitor (configurable vía topology.yaml: monitors.amount). Cada instancia ejecuta dos subsistemas en paralelo:

1. Bully election (TCP): Elige un líder entre los monitores
2. Health-check (UDP): Solo el líder hace ping a los workers

Además, cada worker (no monitor) también ejecuta un listener UDP pasivo que responde a los pings del monitor líder.

En el Bully election el propósito es elegir un único monitor líder que sea el responsable de hacer health-checks. Si el líder cae, los demás monitores detectan la falta de pings y ejecutan una nueva elección.

En el Health-check el monitor líder se encarga de verificar periódicamente que todos los workers estén vivos, mediante comunicación UDP, con un protocolo de request-response.

Entonces el flujo completo sería:

- Monitores eligen líder via bully TCP
- Y solo el líder:
 - Cada ping_interval (ej: 5s): Envía ping a todos los workers
 - Espera respuesta con auto-descripción
 - Si responde, resetea contador de fallos
 - Si NO responde, incrementa contador
 - Si contador $\geq$ threshold (ej: 3): docker start <container_name>
 - Y repite

Manejo de Fallas del Gateway

Estando en un entorno donde pueden ocurrir fallas, podría pasar que el que falle no sea un worker, sino que sea el gateway. Lo importante a tener en cuenta es que el gateway es el que mantiene la conexión con los clientes, y, al caerse, esa conexión se pierde, por lo que se decidió que si el gateway se cae los clientes serán desconectados y, si quieren obtener los resultados, deberán volver a conectarse y ser procesados como un cliente nuevo volviendo a enviar todas las transacciones a analizar.

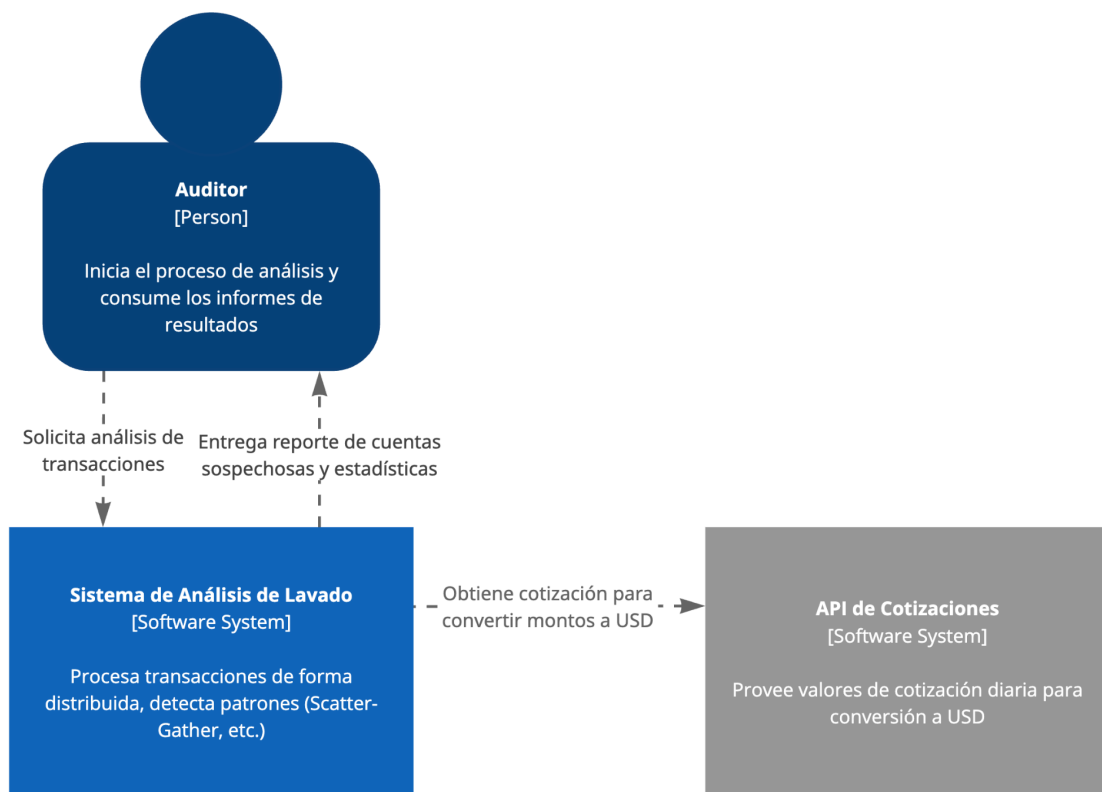
Teniendo en cuenta lo anterior, ahora el problema es limpiar el estado de estos clientes en los workers, que limpian el estado al recibir el EOF del cliente. Para ello se utiliza la misma lógica que se usa cuando un cliente se desconecta, el gateway introduce en el sistema un EOF de estos clientes.

Pero ¿Como el gateway que acaba de instanciarse sabe que clientes estaban conectados a la instancia de gateway anterior cuando falló? Por esto se introdujo que el gateway persista en disco los clientes que están conectados, entonces, cuando una nueva instancia del gateway es levantada por algún monitor, lo primero que hace es revisar esta lista de clientes conectados antes de la caída e inyectar en el sistema un EOF para cada uno de esos clientes, para después descartar todos los resultados recibidos para esos clientes.

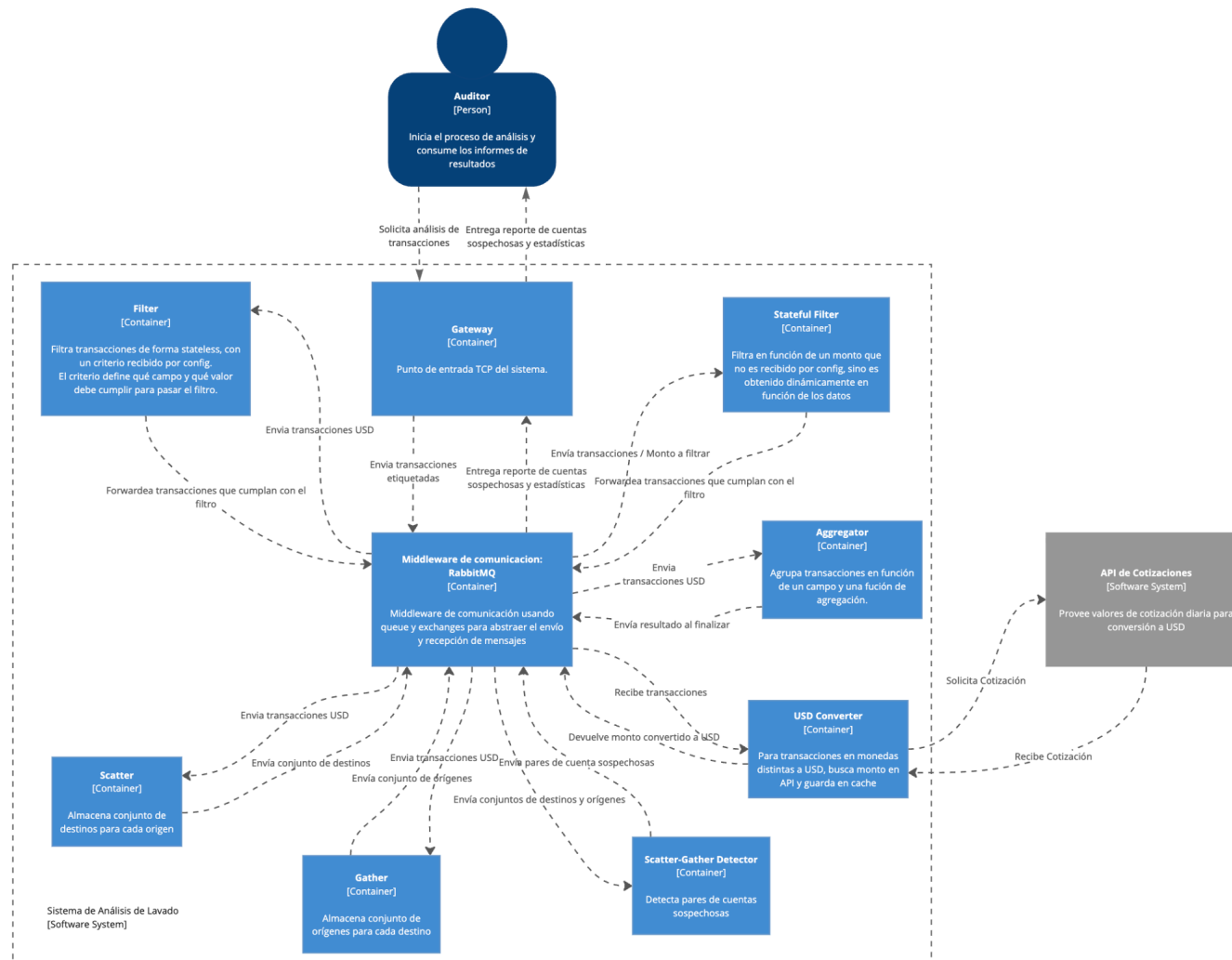
Diagramas

Vista C4

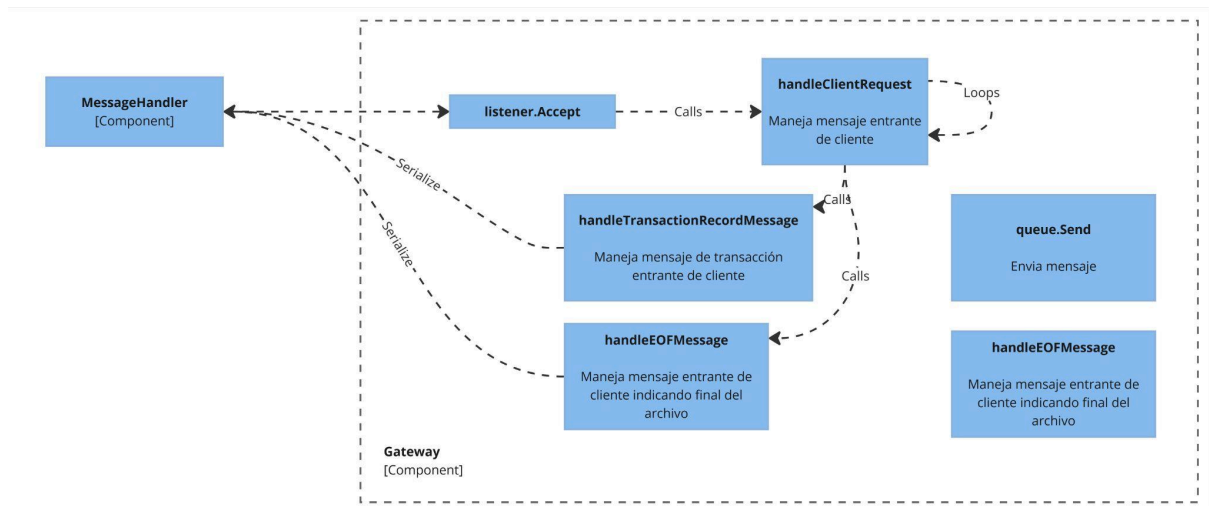
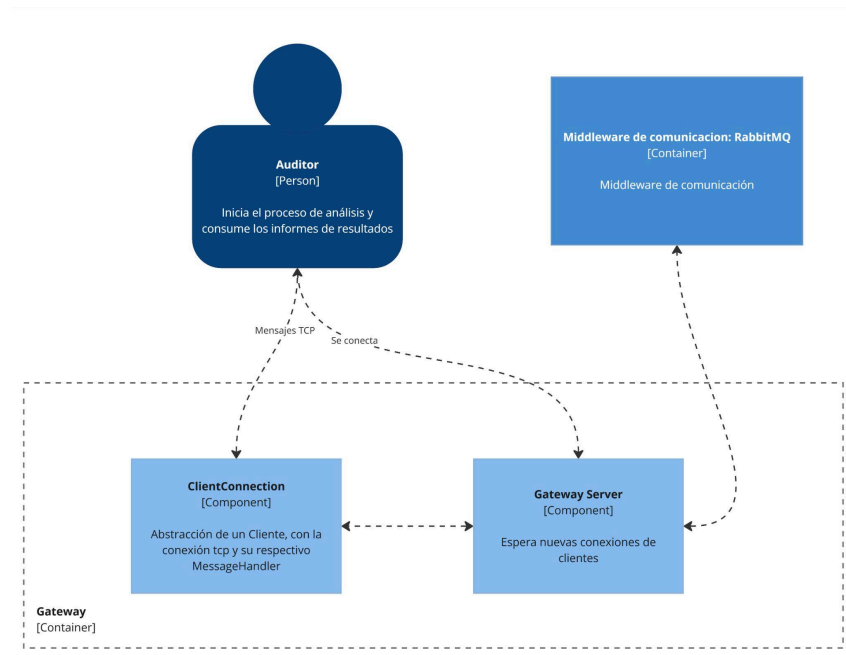
Context



Container



Components



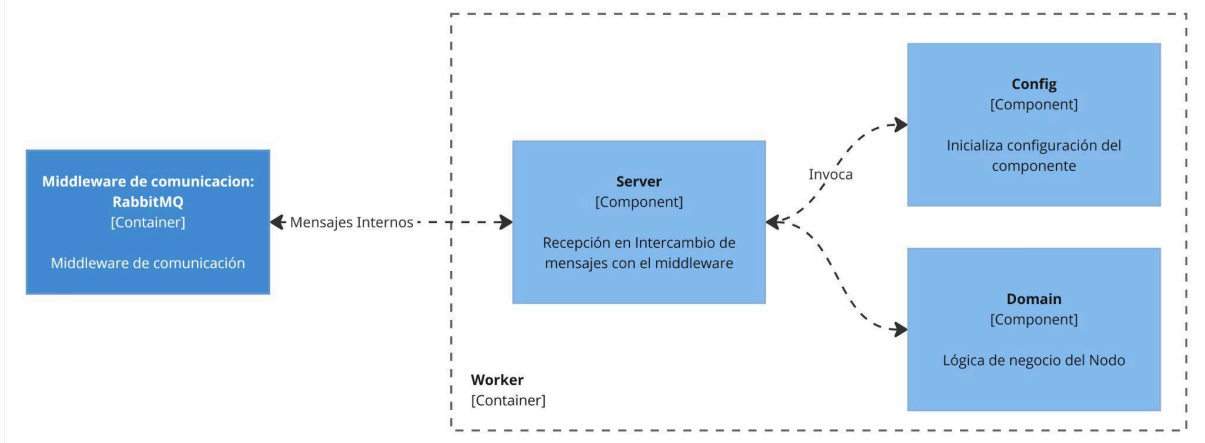
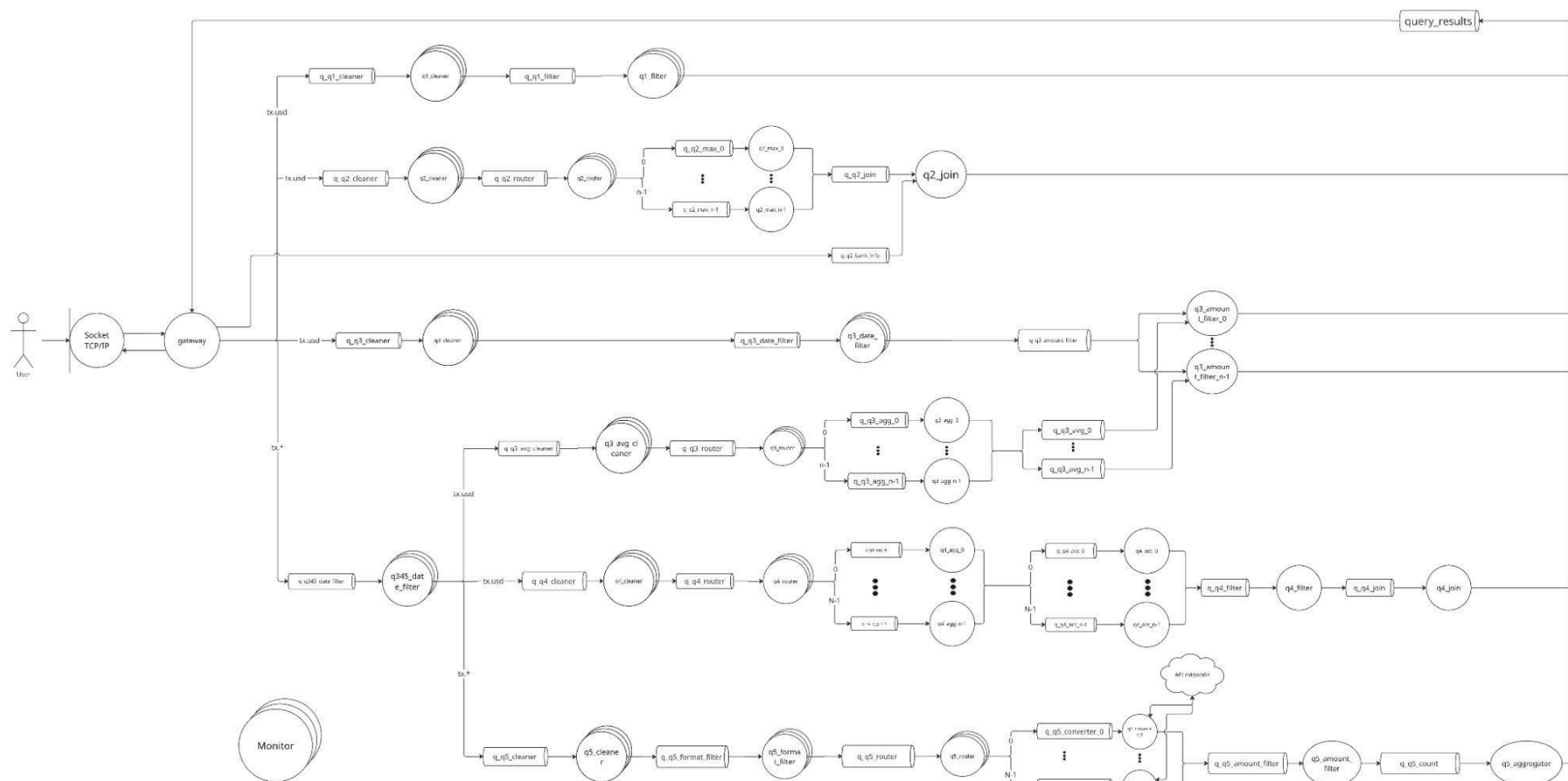


Diagrama de Robustez



Despliegue

Actividades

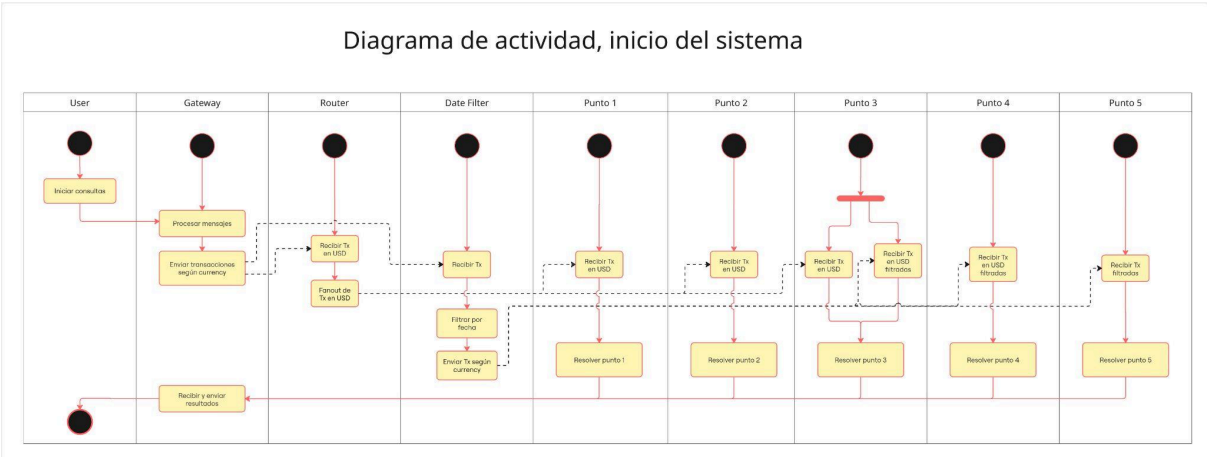


Diagrama de actividad, punto 1

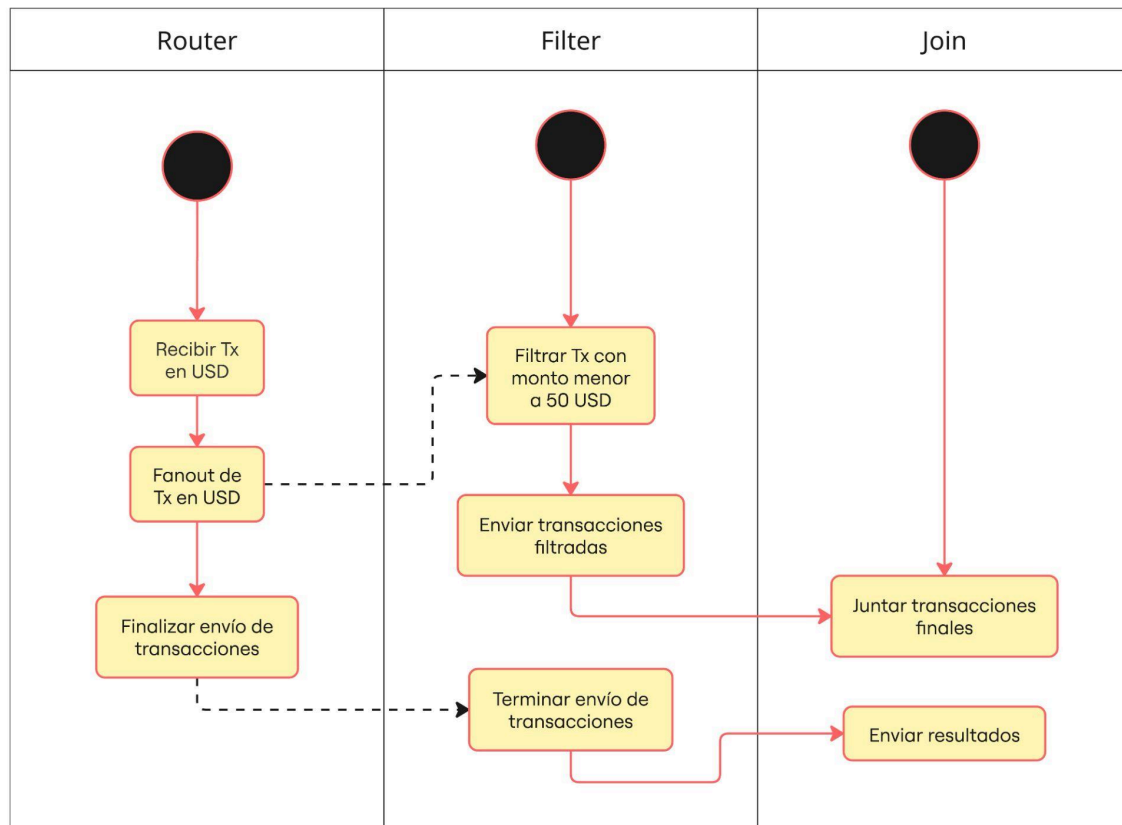


Diagrama de actividad, punto 2

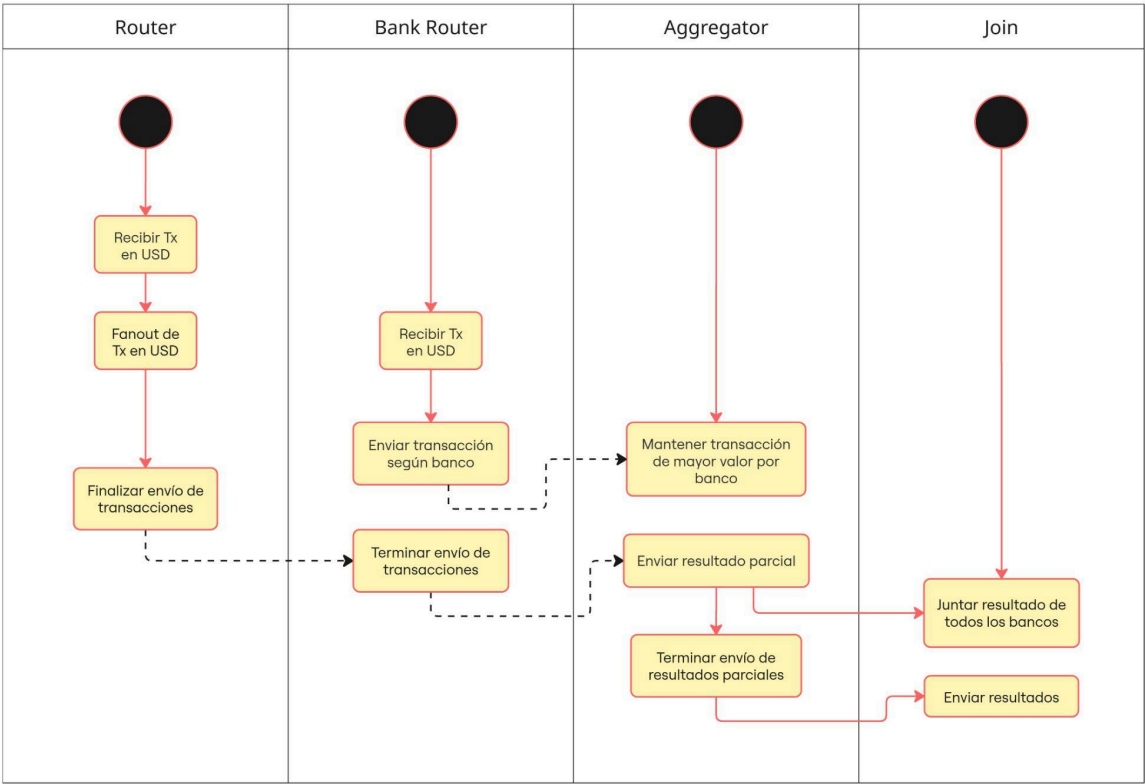


Diagrama de actividad, punto 3

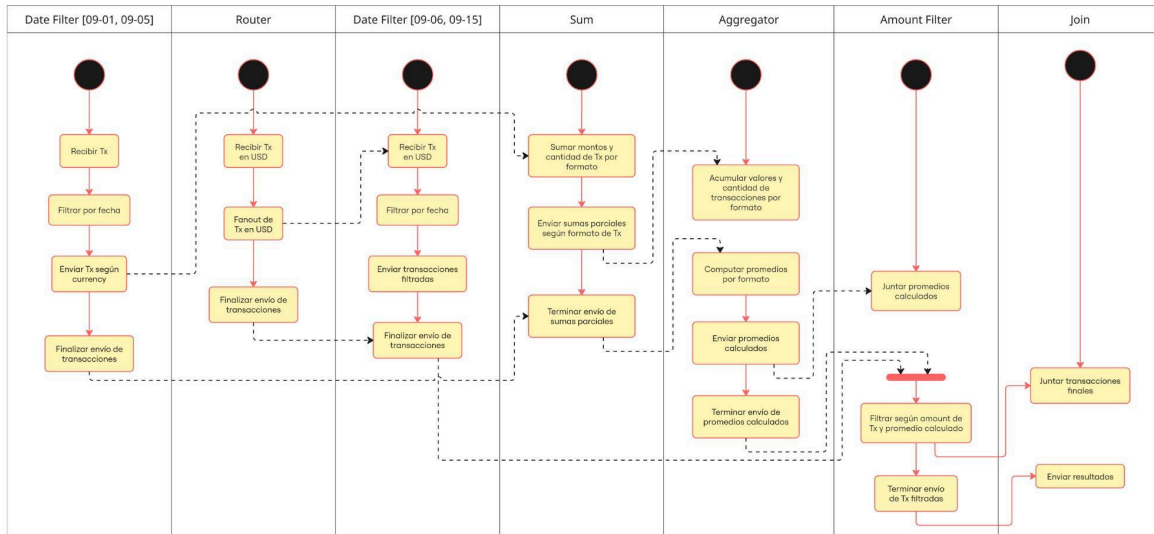


Diagrama de actividad, punto 4

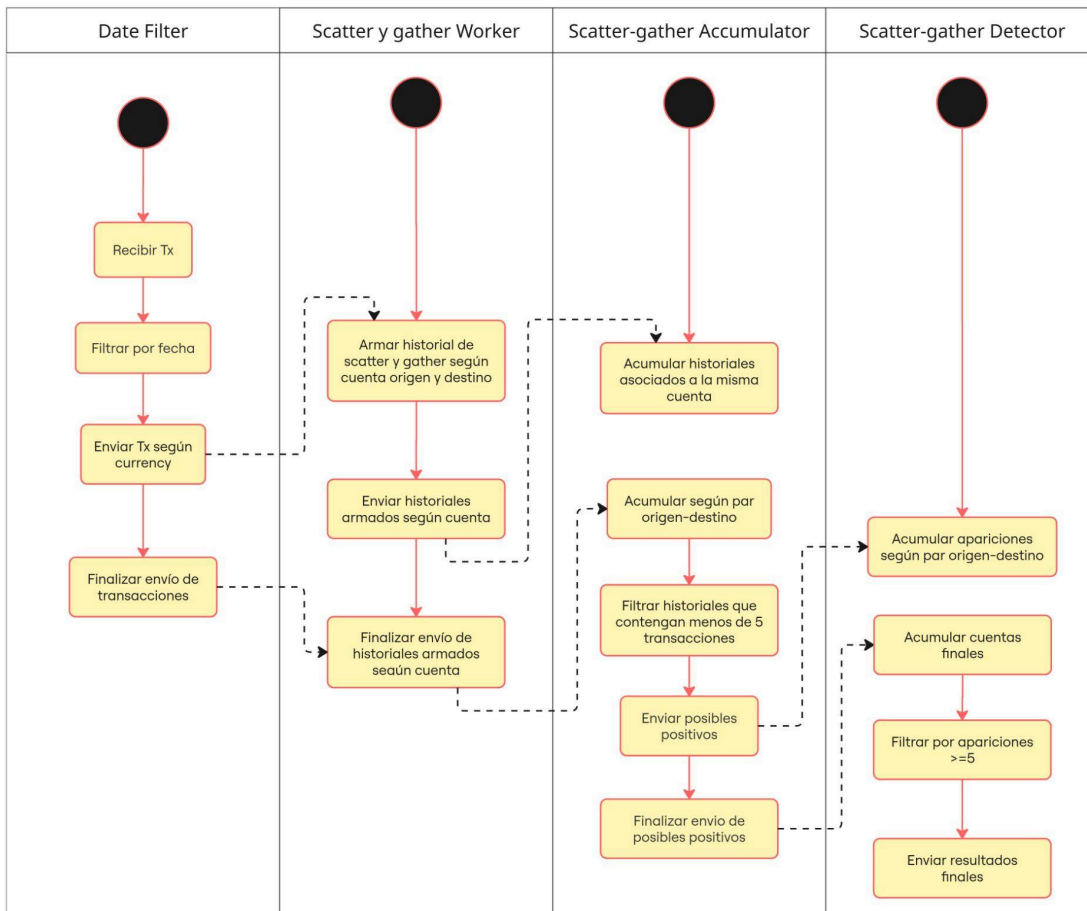
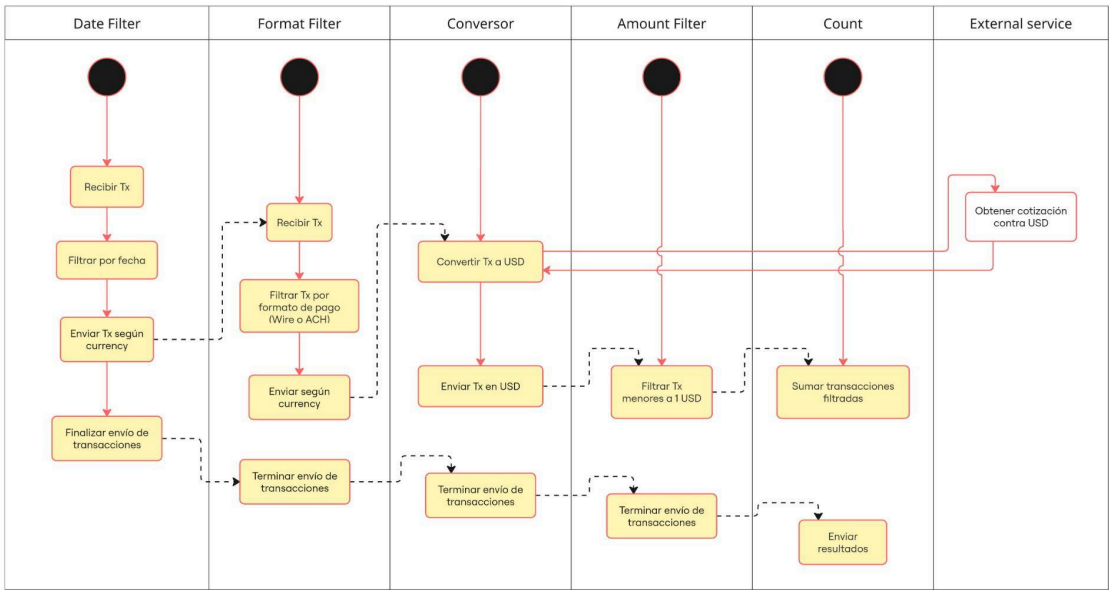
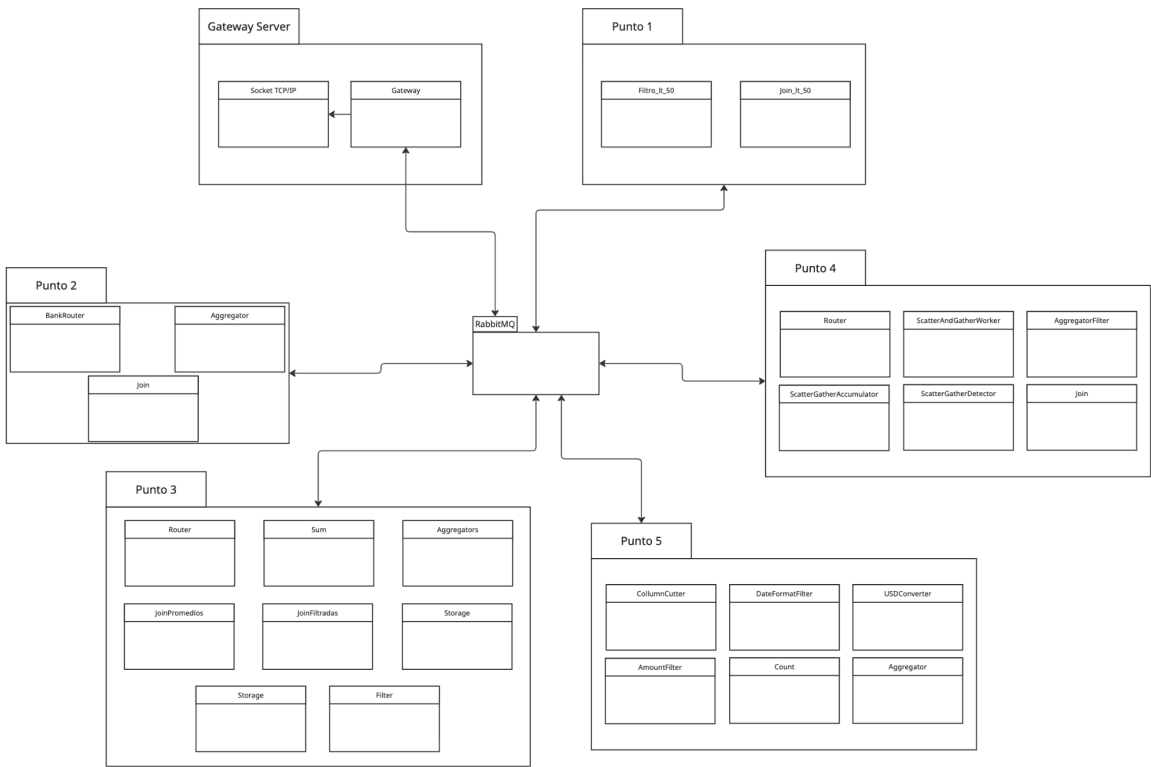


Diagrama de actividad, punto 5

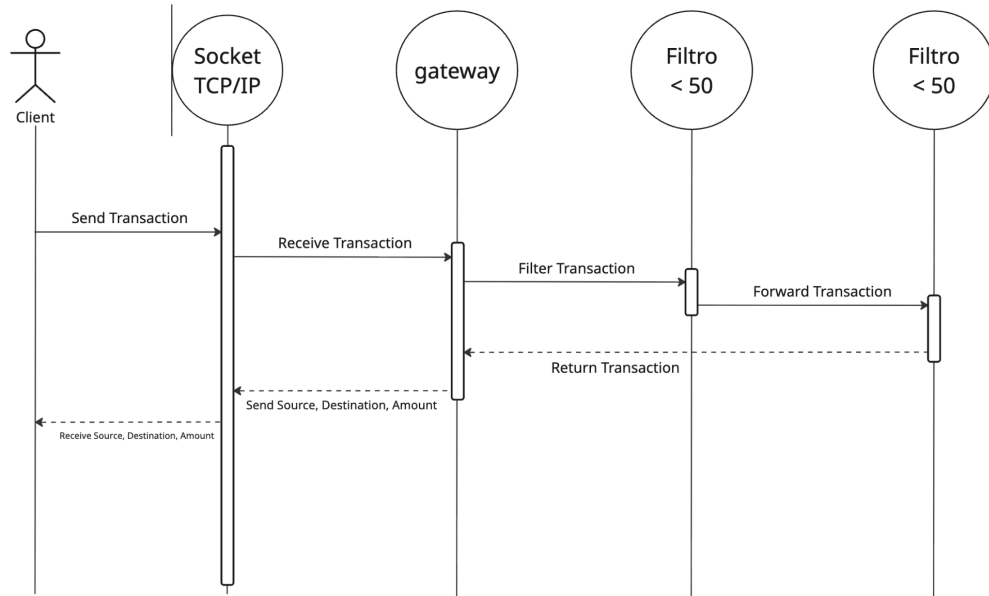


Paquetes

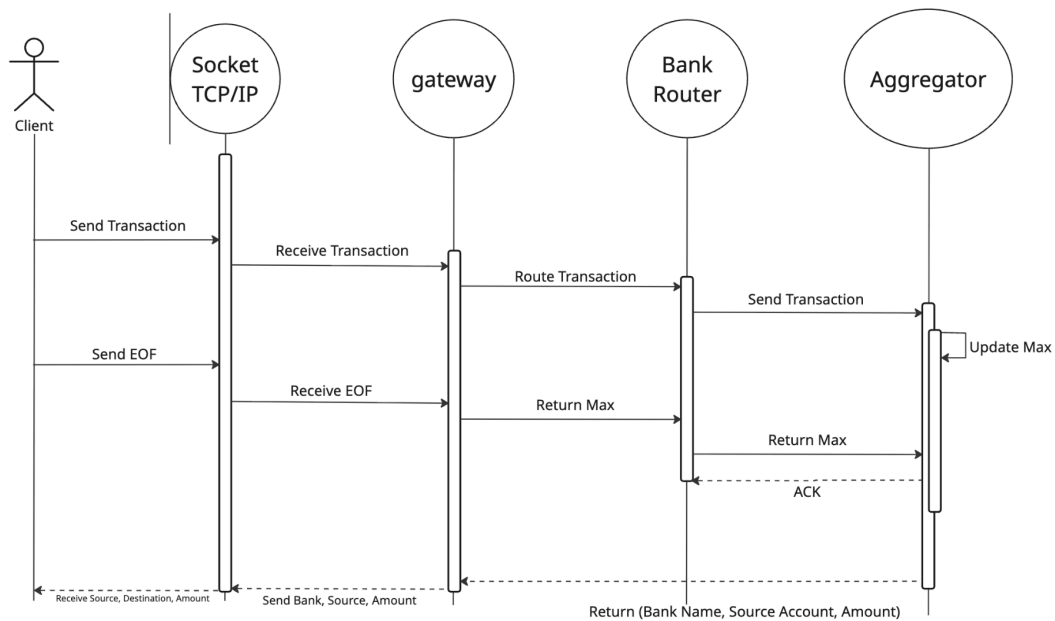


Secuencia

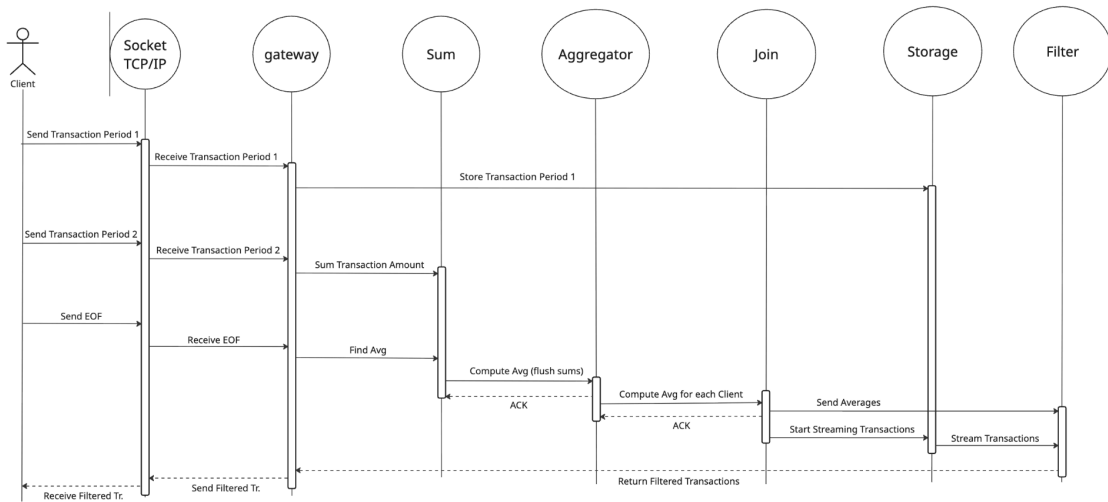
Punto 1: Transacciones menores a 50 USD



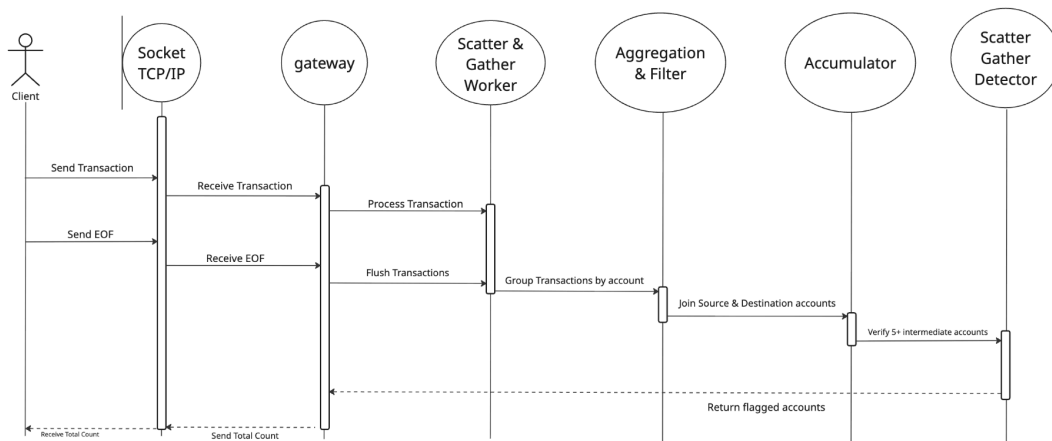
Punto 2: Transacción de monto máximo USD para cada banco



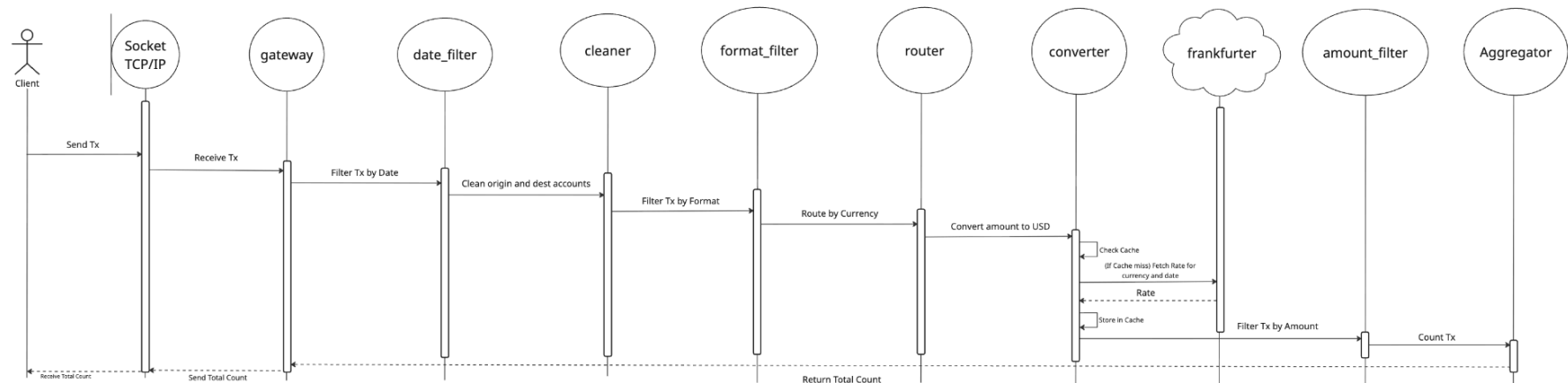
Punto 3: Transacciones de período 1 de monto menor a 1% del promedio de período 2



Punto 4: Cuentas que cumplen con patrón Scatter Gather



Punto 5: Cant. de transacciones filtradas por rango de fechas, formato y monto

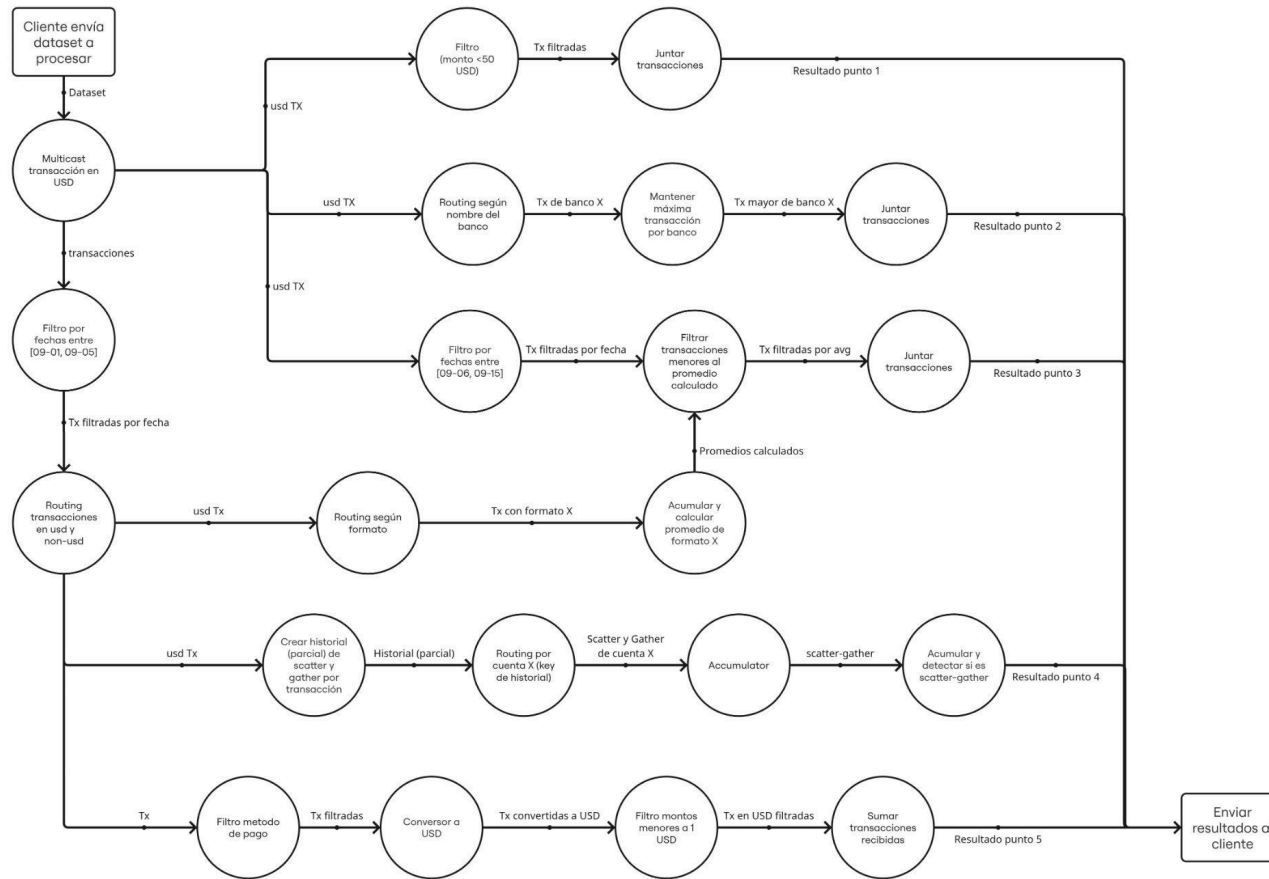


Para el punto 5, filtramos las transacciones por fecha como en las queries 3, 4 y 5, y limpiamos las columnas que no son más necesarias de este punto en adelante: From Account, From Bank, To Account, To Bank. Luego, un filter las filtra por formato (Que sea 'ACH' o 'Wire'), y luego un Router sharea por currency y decide a cuál de los converter siguientes enviar la transacción, para que la misma currency vaya siempre al mismo nodo y así distribuir las transacciones entre los converter.

El converter va a realizar la conversión a USD usando la api de Frankfurter si es necesario o posible (si es Bitcoin, o la currency no es soportada por Frankfurter, usa conversiones hardcodeadas). Luego las transacciones son filtradas por monto y luego acumuladas por el aggregator que cuenta cuantas transacciones recibe

DAG

Diagrama DAG del sistema



División de tareas

Sección	Ezequiel Lazarte	Ignacio Vetrano	Manuel Rivera Villatte
Infraestructura	Armado de script para generar docker-compose a partir de una topología		Protocolo
Middleware	Abstracción de RabbitMQ		
Workers	Punto 4	Punto 3, Planificación de abstracción de workers	Punto 1, 2, 4, 5
Escalabilidad		Coordinación Workers	
Pruebas	Chaos Monkey		Automatizar validación de outputs
Tolerancia	Monitores	Algoritmo de coordinación tolerante a fallos	IDs de mensajes, persistencia

